

1. Знакомство с архитектурой React Vite проекта

Цель занятия

На этом занятии мы разбираем готовый шаблонный проект на React и Vite. Главная задача - понять, из каких файлов состоит проект, за что отвечает каждый файл и как браузер в итоге показывает пользователю страницу.

После занятия должны уметь:

- объяснить, зачем нужен Vite в React-проекте;
- найти главный HTML-файл приложения;
- понять, какой JavaScript-файл запускается первым;
- объяснить, как компонент `App` попадает на страницу;
- отличать файлы исходного кода от служебных файлов проекта;
- понимать, где лежат изображения, стили и настройки.

1. Что такое React Vite проект

React - это библиотека для создания пользовательских интерфейсов. С ее помощью страницу удобно разбивать на компоненты: кнопки, карточки, блоки, формы, меню и другие части интерфейса.

Vite - это инструмент разработки. Он помогает запускать проект локально, быстро обновлять страницу при изменении кода и собирать готовую версию проекта для публикации.

Если объяснить проще:

- React отвечает за интерфейс;
- Vite отвечает за запуск, сборку и удобную разработку;
- браузер показывает результат пользователю.

2. Структура проекта

В проекте находятся такие основные файлы и папки:

text

```
sample/
├── public/
│   ├── favicon.svg
│   └── icons.svg
├── src/
│   ├── assets/
│   │   ├── hero.png
│   │   ├── react.svg
│   │   └── vite.svg
│   ├── App.css
│   ├── App.jsx
│   ├── index.css
│   └── main.jsx
├── .gitignore
├── eslint.config.js
├── index.html
├── package-lock.json
├── package.json
├── README.md
└── vite.config.js
```

Сразу важно понять: не все файлы одинаково часто редактируются. Вы чаще всего будете работать с файлами в папке `src`, иногда с `public`, а конфигурационные файлы обычно меняются реже.

3. Краткая таблица файлов

Файл или папка	За что отвечает
<code>index.html</code>	Главная HTML-страница. В ней есть контейнер <code>root</code> , куда React вставляет приложение.
<code>src/main.jsx</code>	Главная точка входа JavaScript. Запускает React-приложение.
<code>src/App.jsx</code>	Основной React-компонент приложения. Сейчас здесь находится главный интерфейс страницы.
<code>src/index.css</code>	Глобальные стили для всей страницы: шрифты, цвета, фон, стили <code>body</code> , <code>#root</code> , заголовков.
<code>src/App.css</code>	Стили конкретно для компонента <code>App</code> : кнопка, hero-блок, секции, адаптивность.
<code>src/assets/</code>	Изображения, которые импортируются прямо в JavaScript/JSX-код.
<code>public/</code>	Статические файлы, доступные напрямую из браузера по пути от корня сайта.
<code>package.json</code>	Описание проекта: команды запуска, зависимости, версия, тип модулей.
<code>package-lock.json</code>	Точный список установленных npm-пакетов и их версий.
<code>vite.config.js</code>	Настройка Vite. В этом проекте подключает React-плагин.
<code>eslint.config.js</code>	Настройка ESLint - инструмента для проверки качества кода.
<code>.gitignore</code>	Список файлов и папок, которые не нужно добавлять в Git.
<code>README.md</code>	Краткое описание проекта.

4. Как приложение запускается в браузере

Когда мы запускаем проект командой:

```
bash
  npm run dev
```

npm смотрит в файл `package.json` и находит там команду `dev`:

```
json
```

```
{
  "scripts": {
    "dev": "vite",
    "build": "vite build",
    "lint": "eslint .",
    "preview": "vite preview"
  }
}
```

Команда `npm run dev` фактически запускает Vite. Vite поднимает локальный сервер разработки и отдает браузеру файл `index.html`.

Дальше происходит цепочка:

```
text
  npm run dev
    ↓
  Vite запускает сервер разработки
    ↓
  Браузер открывает index.html
    ↓
  index.html подключает /src/main.jsx
    ↓
  main.jsx подключает App.jsx
    ↓
  React вставляет компонент App в div#root
```

5. Файл `index.html`

Файл `index.html` - это HTML-основа приложения. Он не содержит весь интерфейс сайта, как в обычной статической верстке. Вместо этого он содержит место, куда React вставит приложение.

```
html
<body>
  <div id="root"></div>
  <script type="module" src="/src/main.jsx"></script>
</body>
```

Здесь есть две самые важные строки.

Первая:

```
html
<div id="root"></div>
```

Это пустой контейнер. Пока React не запущен, внутри него ничего нет. После запуска React добавляет туда интерфейс приложения.

Вторая:

```
html
<script type="module" src="/src/main.jsx"></script>
```

Эта строка говорит браузеру: загрузи JavaScript-файл `src/main.jsx`. Именно с него начинается выполнение нашего приложения.

Также в `index.html` есть служебные настройки:

```
html
```

```
<meta charset="UTF-8" />
<meta name="viewport" content="width=device-width, initial-scale=1.0" />
<link rel="icon" type="image/svg+xml" href="/favicon.svg" />
```

charset="UTF-8" нужен для правильного отображения символов. viewport помогает странице нормально выглядеть на телефонах. favicon.svg задает иконку вкладки браузера.

6. Файл `src/main.jsx`

Файл main.jsx - точка входа React-приложения.

```
javascript
import { StrictMode } from 'react'
import { createRoot } from 'react-dom/client'
import './index.css'
import App from './App.jsx'

createRoot(document.getElementById('root')).render(
  <StrictMode>
    <App />
  </StrictMode>,
)
```

Разберем по частям.

```
javascript
import { StrictMode } from 'react'
```

StrictMode - специальный режим React для разработки. Он помогает находить потенциальные ошибки в компонентах. На внешний вид страницы он напрямую не влияет.

```
javascript
import { createRoot } from 'react-dom/client'
```

createRoot нужен, чтобы связать React с настоящим HTML-элементом на странице.

```
javascript
import './index.css'
```

Так подключаются глобальные стили. После этого стили из index.css применяются ко всей странице.

```
javascript
import App from './App.jsx'
```

Так подключается главный компонент приложения.

Самая важная часть:

```
javascript
createRoot(document.getElementById('root')).render(
  <StrictMode>
    <App />
  </StrictMode>,
)
```

document.getElementById('root') находит элемент:

```
html
<div id="root"></div>
```

Затем React говорит: в этот контейнер нужно отрисовать компонент App.

7. Что такое JSX

В файлах React часто встречается синтаксис, похожий на HTML внутри JavaScript. Он называется JSX.

Пример из App.jsx:

```
javascript
return (
  <>
    <section id="center">
      <h1>Get started</h1>
    </section>
  </>
)
```

JSX похож на HTML, но это не обычный HTML-файл. Это специальная запись, которую Vite и React преобразуют в JavaScript-код.

В JSX есть несколько важных отличий:

- вместо `class` используется `className`;
- JavaScript-выражения вставляются в фигурные скобки `{}`;
- компонент должен возвращать один общий корневой элемент;
- пустой общий элемент можно записать как `<>...</>`.

Пример:

```
javascript
<button className="counter">
  Count is {count}
</button>
```

Здесь `className="counter"` подключает CSS-класс, а `{count}` выводит значение JavaScript-переменной.

8. Файл `src/App.jsx`

App.jsx - главный компонент проекта. Компонент - это функция, которая возвращает JSX.

В начале файла подключаются зависимости:

```
javascript
import { useState } from 'react'
import reactLogo from './assets/react.svg'
import viteLogo from './assets/vite.svg'
import heroImg from './assets/hero.png'
import './App.css'
```

Что здесь происходит:

- `useState` импортируется из React и нужен для хранения состояния;
- `reactLogo`, `viteLogo`, `heroImg` - изображения из папки `src/assets`;
- `App.css` - стили для компонента App.

Главная функция компонента:

```
javascript
function App() {
  const [count, setCount] = useState(0)

  return (
    <>
      {/* JSX-разметка компонента */}
    </>
  )
}
```

Строка:

```
javascript
const [count, setCount] = useState(0)
```

создает состояние. `count` - текущее значение счетчика, а `setCount` - функция для изменения этого значения. Начальное значение счетчика равно 0.

Кнопка счетчика выглядит так:

```
javascript
<button
  type="button"
  className="counter"
  onClick={() => setCount((count) => count + 1)}
>
  Count is {count}
</button>
```

Когда пользователь нажимает на кнопку, срабатывает `onClick`. Он вызывает `setCount`, значение `count` увеличивается на 1, React заново обновляет нужную часть интерфейса.

Важно: React не перезагружает всю страницу. Он обновляет интерфейс там, где изменились данные.

В конце файла находится экспорт:

```
javascript
export default App
```

Эта строка позволяет импортировать компонент в другом файле:

```
javascript
import App from './App.jsx'
```

Именно так `main.jsx` получает доступ к компоненту `App`.

9. Изображения в проекте

В проекте изображения используются двумя способами.

Первый способ - импорт из папки `src/assets`:

```
javascript
import reactLogo from './assets/react.svg'
import viteLogo from './assets/vite.svg'
import heroImg from './assets/hero.png'
```

Потом изображения используются в JSX:

```
javascript
<img src={heroImg} className="base" width="170" height="179" alt="" />
<img src={reactLogo} className="framework" alt="React logo" />
<img src={viteLogo} className="vite" alt="Vite logo" />
```

Такой способ удобен, когда изображение является частью компонента.

Второй способ - файлы из папки `public`. Например, в `App.jsx` используется SVG-спрайт:

```
javascript
<svg className="icon" role="presentation" aria-hidden="true">
  <use href="/icons.svg#documentation-icon"></use>
</svg>
```

Путь начинается с `/icons.svg`, потому что файл `icons.svg` лежит в папке `public`. Все файлы из `public` доступны из корня сайта.

Пример:

```
text
public/icons.svg → /icons.svg
public/favicon.svg → /favicon.svg
```

10. Файл `src/index.css`

`index.css` содержит глобальные стили. Они влияют на всю страницу.

В начале файла объявляются CSS-переменные:

```
css
:root {
  --text: #6b6375;
  --text-h: #08060d;
  --bg: #fff;
  --border: #e5e4e7;
  --accent: #aa3bff;
}
```

CSS-переменная - это именованное значение, которое можно использовать много раз.

Например:

```
css
body {
  color: var(--text);
}
```

Если потом изменить значение `--text`, цвет текста изменится во всех местах, где используется `var(--text)`.

Также в `index.css` задается темная тема:

CSS

```
@media (prefers-color-scheme: dark) {
  :root {
    --text: #9ca3af;
    --text-h: #f3f4f6;
    --bg: #16171d;
  }
}
```

`prefers-color-scheme: dark` означает: если у пользователя в системе включена темная тема, применить эти значения.

Один из самых важных селекторов:

CSS

```
#root {
  width: 1126px;
  max-width: 100%;
  margin: 0 auto;
  text-align: center;
  min-height: 100svh;
  display: flex;
  flex-direction: column;
}
```

Это стили для того самого контейнера `root`, который находится в `index.html`. Получается, что HTML создает контейнер, React вставляет в него приложение, а CSS задает ему внешний вид.

11. Файл ``src/App.css``

`App.css` содержит стили для интерфейса из `App.jsx`.

Например, в JSX есть кнопка:

Javascript

```
<button type="button" className="counter">
  Count is {count}
</button>
```

А в CSS есть стиль для класса `counter`:

CSS

```
.counter {
  font-size: 16px;
  padding: 5px 10px;
  border-radius: 5px;
  color: var(--accent);
  background: var(--accent-bg);
}
```

Связь простая:

text

`className="counter"` в JSX

↓

`.counter` в CSS

↓

браузер применяет стили к кнопке

В проекте используется современная CSS-вложенность:

CSS

```
.counter {
  border: 2px solid transparent;

  &:hover {
    border-color: var(--accent-border);
  }
}
```

Это означает: когда пользователь наводит мышь на элемент `.counter`, поменять цвет рамки.

В обычной CSS-записи это выглядело бы так:

CSS

```
.counter:hover {
  border-color: var(--accent-border);
}
```

Еще один пример вложенности:

CSS

```
.hero {
  position: relative;

  .base,
  .framework,
  .vite {
    inset-inline: 0;
    margin: 0 auto;
  }
}
```

Это значит: стили `.base`, `.framework` и `.vite` применяются к элементам, которые находятся внутри `.hero`.

12. Папка `src/assets`

Папка `src/assets` хранит файлы, которые используются внутри компонентов через `import`.

В проекте там лежат:

text

```
src/assets/
├─ hero.png
├─ react.svg
└─ vite.svg
```

Пример использования:

javascript

```
import heroImg from './assets/hero.png'

function App() {
  return <img src={heroImg} alt="" />
}
```

Почему нельзя просто написать путь строкой? В Vite можно использовать оба подхода, но импорт удобен тем, что сборщик видит зависимость, проверяет файл и правильно подготавливает его для итоговой сборки.

13. Папка `public`

Папка `public` нужна для файлов, которые должны быть доступны напрямую.

В проекте там лежат:

```
text
  public/
  ├── favicon.svg
  └── icons.svg
```

Файл `favicon.svg` подключается в `index.html`:

```
html
  <link rel="icon" type="image/svg+xml" href="/favicon.svg" />
```

Файл `icons.svg` используется в `App.jsx`:

```
javascript
  <use href="/icons.svg#github-icon"></use>
```

Главное правило:

```
text
  Если файл лежит в public, обращаемся к нему от корня сайта: /имя-файла
```

14. Файл `package.json`

`package.json` - паспорт проекта. Он содержит имя проекта, команды и зависимости.

Фрагмент:

```
json
{
  "name": "sample",
  "private": true,
  "version": "0.0.0",
  "type": "module"
}
```

"type": "module" означает, что проект использует современную систему модулей JavaScript: `import` и `export`.

Команды проекта:

```
json
{
  "scripts": {
    "dev": "vite",
    "build": "vite build",
    "lint": "eslint .",
    "preview": "vite preview"
  }
}
```

Обычно используются такие команды:

```
bash
  npm run dev
```

Запускает проект для разработки.

```
bash
  npm run build
```

Создает готовую версию проекта в папке `dist`.

```
bash
npm run lint
```

Проверяет код с помощью ESLint. (об этом позже)

```
bash
npm run preview
```

Позволяет посмотреть собранную версию проекта локально.

Зависимости:

```
json
{
  "dependencies": {
    "react": "^19.2.6",
    "react-dom": "^19.2.6"
  }
}
```

`react` нужен для создания компонентов. `react-dom` нужен, чтобы React мог работать с DOM браузера.

Dev-зависимости:

```
json
{
  "devDependencies": {
    "@vitejs/plugin-react": "^6.0.1",
    "vite": "^8.0.12",
    "eslint": "^10.3.0"
  }
}
```

Dev-зависимости нужны во время разработки, но обычно не попадают как отдельные библиотеки в итоговый пользовательский интерфейс.

15. Файл `vite.config.js`

`vite.config.js` содержит настройки Vite.

```
javascript
import { defineConfig } from 'vite'
import react from '@vitejs/plugin-react'

export default defineConfig({
  plugins: [react()],
})
```

Здесь подключается React-плагин. Он помогает Vite понимать React, JSX и особенности разработки React-приложений.

На первом этапе важно запомнить: если проект создан через шаблон Vite React, этот файл уже настроен, и обычно его не нужно менять в первых практических работах.

16. Файл `eslint.config.js`

ESLint - это инструмент, который проверяет код и помогает находить ошибки или плохие практики.

В проекте настроена проверка JavaScript и JSX-файлов:

```
javascript
export default defineConfig([
  globalIgnores(['dist']),
  {
    files: ['**/*.{js,jsx}'],
    extends: [
      js.configs.recommended,
      reactHooks.configs.flat.recommended,
      reactRefresh.configs.vite,
    ],
  },
])
```

Важные моменты:

- `globalIgnores(['dist'])` говорит ESLint не проверять папку `dist`;
- `files: ['**/*.{js,jsx}']` означает, что проверяются файлы `.js` и `.jsx`;
- настройки React Hooks помогают правильно использовать хуки, например `useState`;
- настройки React Refresh помогают корректно работать быстрому обновлению страницы во время разработки.

Запуск проверки:

```
bash
npm run lint
```

17. Файл `package-lock.json`

`package-lock.json` создается прм автоматически. Он фиксирует точные версии всех установленных пакетов.

Например, в `package.json` может быть написано:

```
json
  "vite": "^8.0.12"
```

Символ `^` означает, что прм может поставить совместимую более новую версию. А `package-lock.json` запоминает, какая версия была установлена именно сейчас.

Обычно этот файл не редактируют вручную.

18. Файл `.gitignore`

`.gitignore` сообщает Git, какие файлы и папки не нужно сохранять в репозитории.

Обычно туда добавляют:

```
text
node_modules
dist
.env
```

`node_modules` не сохраняют в Git, потому что эту папку можно заново восстановить командой:

```
bash
npm install
```

`dist` не сохраняют, потому что это результат сборки, который можно заново создать командой:

```
bash
npm run build
```

19. Файл `README.md`

`README.md` - файл с описанием проекта. В нем обычно пишут:

- что это за проект;
- как его установить;
- как запустить;
- какие команды доступны;
- какие технологии используются.

В реальной разработке это важный файл для команды.

20. Как файлы взаимодействуют друг с другом

Можно представить проект как цепочку:

```
text
index.html
├─ подключает src/main.jsx
│   ├─ подключает src/index.css
│   └─ подключает src/App.jsx
│       ├─ подключает src/App.css
│       ├─ подключает src/assets/hero.png
│       ├─ подключает src/assets/react.svg
│       └─ подключает src/assets/vite.svg
public/favicon.svg подключается из index.html
public/icons.svg используется напрямую в App.jsx
```

Главная мысль: `index.html` запускает приложение, `main.jsx` подключает React к странице, `App.jsx` описывает интерфейс, а CSS-файлы отвечают за внешний вид.

21. Пример изменения компонента

Допустим, нужно изменить заголовок на странице. В `App.jsx` есть строка:

```
javascript
<h1>Get started</h1>
```

Можно заменить ее на:

```
javascript
<h1>Мое первое React-приложение</h1>
```

После сохранения файла Vite обновит страницу в браузере. Это называется HMR - Hot Module Replacement, горячая замена модулей. Она позволяет видеть изменения почти сразу, без ручной перезагрузки страницы.

22. Пример добавления нового текста

Внутри секции `center` можно добавить новый абзац:

```
javascript
<section id="center">
  <div className="hero">
    <img src={heroImg} className="base" width="170" height="179" alt="" />
    <img src={reactLogo} className="framework" alt="React logo" />
    <img src={viteLogo} className="vite" alt="Vite logo" />
  </div>

  <div>
    <h1>Get started</h1>
    <p>
      Edit <code>src/App.jsx</code> and save to test <code>HMR</code>
    </p>
    <p>Это учебный проект на React и Vite.</p>
  </div>
</section>
```

Если текст добавлен внутри JSX, он появится на странице.

23. Пример добавления стиля

Допустим, новому абзацу нужно задать класс:

```
javascript
<p className="lesson-note">Это учебный проект на React и Vite.</p>
```

Тогда в `App.css` можно добавить стиль:

```
css
.lesson-note {
  color: var(--accent);
  font-weight: 500;
}
```

Связь снова идет через `className`:

```
text
className="lesson-note"
↓
.lesson-note в App.css
↓
новый внешний вид абзаца
```

24. Частые ошибки

Ошибка 1. Написать `class` вместо `className`

Неправильно:

```
javascript
<button class="counter">Count</button>
```

Правильно:

```
javascript
<button className="counter">Count</button>
```

Ошибка 2. Забыть закрыть тег

Неправильно:

```
javascript
<img src={heroImg}>
```

Правильно:

```
javascript
<img src={heroImg} alt="" />
```

Ошибка 3. Использовать переменную без фигурных скобок

Неправильно:

```
javascript

```

Правильно:

```
javascript
<img src={heroImg} alt="" />
```

В первом случае браузер будет искать файл с буквальным именем `heroImg`. Во втором случае React подставит значение переменной `heroImg`.

Ошибка 4. Удалить `export default App`

Если удалить:

```
javascript
export default App
```

то файл `main.jsx` больше не сможет импортировать компонент:

```
javascript
import App from './App.jsx'
```

Приложение перестанет запускаться.

Ошибка 5. Удалить `div id="root"`

Если удалить из `index.html`:

```
html
<div id="root"></div>
```

то строка в `main.jsx`:

```
javascript
document.getElementById('root')
```

не найдет контейнер. React не будет знать, куда вставлять приложение.

25. Мини-практика

Выполните задания по очереди.

Задание 1

Найдите файл `src/App.jsx` и измените заголовок:

```
javascript
<h1>Get started</h1>
```

на:

```
javascript
<h1>Привет, React!</h1>
```

Задание 2

Добавьте под заголовком новый абзац:

```
javascript
<p>Сегодня мы изучаем архитектуру React Vite проекта.</p>
```

Задание 3

Добавьте абзацу класс:

```
javascript
<p className="intro-text">
  Сегодня мы изучаем архитектуру React Vite проекта.
</p>
```

Задание 4

В файле `src/App.css` добавьте стиль:

```
css
.intro-text {
  color: var(--accent);
  font-weight: 500;
}
```

Задание 5

Нажмите кнопку счетчика несколько раз и объясните своими словами, почему число на кнопке меняется.

26. Контрольные вопросы

1. Какой файл первым открывает браузер в Vite-проекте?
2. Зачем нужен элемент `<div id="root"></div>`?
3. Какой файл является точкой входа React-приложения?
4. Что делает `createRoot`?
5. Зачем в `main.jsx` импортируется `App.jsx`?
6. Что такое компонент в React?
7. Чем JSX отличается от обычного HTML?
8. Почему в JSX используется `className`, а не `class`?
9. Для чего нужен `useState`?
10. Чем папка `src/assets` отличается от папки `public`?
11. Где находятся глобальные стили проекта?
12. Где находятся стили компонента `App`?
13. За что отвечает `package.json`?
14. Какую команду нужно выполнить, чтобы запустить проект?
15. Что делает команда `npm run build`?

27. Краткий итог

React Vite проект состоит из нескольких важных частей. `index.html` содержит HTML-контейнер для приложения. `main.jsx` запускает React и подключает главный компонент. `App.jsx` описывает интерфейс. CSS-файлы отвечают за внешний вид. Папка `src/assets` хранит изображения, которые импортируются в компоненты, а `public` хранит файлы, доступные напрямую.

Главная цепочка проекта:

text

`index.html` → `src/main.jsx` → `src/App.jsx` → интерфейс в браузере